PATRONES DE DISEÑO

1. ¿Qué son?

Soluciones reutilizables y probadas a problemas de diseño recurrentes. No son código listo, sino plantillas que guían la estructura y colaboración entre objetos.

2. Los patrones de diseño no son frameworks.

Un patrón es una idea/plantilla para resolver un problema recurrente; un framework (React, Angular, Express, Django…) es un conjunto de librerías/opiniones para construir apps. Los frameworks a veces usan patrones por dentro y se puede aplicar en código.

3. Clasificaciones principales de patrones de diseño

A. Creacionales: cómo crear objetos sin acoplar el código a clases concretas.

B. Estructurales: cómo componer clases/objetos para formar estructuras flexibles.

C. De comportamiento: cómo colaboran los objetos y reparten responsabilidades.

A. Creacionales

Singleton

Garantiza una sola instancia y un punto global de acceso.
Cuándo: recursos únicos (config, pool).

```
class Config:
    _inst = None
    def __new__(cls):
        if not cls._inst: cls._inst = super().__new__(cls)
        return cls._inst
```

Factory Method

Delega a subclases la creación de objetos compatibles.
Cuándo: necesitas familias de productos intercambiables.

```
class ParserFactory:
    def create(fmt):
        return JsonParser() if fmt=="json" else XmlParser()
```

Builder

Separa la construcción paso a paso de la representación final.
Cuándo: objetos complejos con muchas opciones.

```
order = (OrderBuilder()
    .with_item("Laptop").with_coupon("SALE10")
    .deliver_to("Av. 123").build())
```

B. Estructurales

Adapter

Convierte una interfaz existente a la que el cliente espera.
Cuándo: integrar librerías/servicios con API distinta.

```
class StripeAdapter(Payments):
  def pay(self, amount): return stripe.charge(total=amount)
```

Decorator

Añade responsabilidades dinámicamente sin herencia.
Cuándo: sumar funcionalidades opt-in (logging, cache).

```
def cached(repo):
  def get(id):
    if id in cache: return cache[id]
    cache[id]=repo.get(id); return cache[id]
  return type("RepoCache",(object,),{"get":get})( )
```

Facade

Ofrece una interfaz simple sobre un subsistema complejo.
Cuándo: simplificar uso de módulos grandes.

```
class VideoFacade:
  def convert(self, f): demux(); transcode(); mux()
```

Composite

Trata objetos individuales y compuestos de forma uniforme.
Cuándo: árboles (UI, carpetas).

```
class Node:
    def render(self): pass
class Leaf(Node): ...
class Group(Node):
    def __init__(self): self.children=[]
    def render(self): [c.render() for c in self.children]
```

C. De comportamiento

Strategy

Encapsula algoritmos intercambiables.
Cuándo: elegir comportamiento en tiempo de ejecución.

```
def price(base, strategy): return strategy(base)
price(100, lambda x: x*0.9)   # descuento
```

Observer

Publica–suscribe: muchos oyentes reaccionan a cambios.
Cuándo: eventos, UIs, cache invalidation.

```
class Event:
    subs=[];
    def emit(self,data): [s(data) for s in self.subs]
```

Command

Encapsula una petición como objeto (deshacer, colas).
Cuándo: redo/undo, jobs.

```
class RenameCmd:
    def __init__(s,fs,a,b): s.fs, s.a, s.b = fs,a,b
    def exec(s): s.fs.rename(s.a,s.b)
```

State

Cambia comportamiento según estado interno sin ifs gigantes.
Cuándo: máquinas de estados (pedido, auth).

```
class Order:
    def __init__(s): s.state=Pending()
    def pay(s): s.state = s.state.pay()
```

4. **Ejemplo en Frontend y Backend**

Frontend

1) Observer (pub/sub)

Idea simple: cuando algo cambia, muchos escuchas se enteran automáticamente.
Para qué sirve: mantener la interfaz sincronizada con el estado.
Cómo se ve:

- React + Redux: componentes que se suscriben al store y se vuelven a renderizar si cambia.
- Vue: la reactividad detecta cambios y actualiza lo visible.
- DOM: addEventListener escucha eventos (click, input, etc.).
  Señales de uso: haces setState o cambias datos y varias partes de la UI deben actualizarse.
  Cuidado: demasiados listeners desordenados → difícil de depurar. Centraliza eventos.

2) Strategy

Idea simple: elegir cómo hacer algo sin escribir un "mega-if" gigante.
Para qué sirve: cambiar reglas/algoritmos según contexto (locale, plan de precios, modo claro/oscuro).
Cómo se ve: seleccionar un formateador de moneda o fecha según el idioma del usuario.

3) State

Idea simple: un componente cambia su comportamiento según su estado.
Para qué sirve: modelar pantallas con pasos: idle → cargando → éxito/error.
Cómo se ve: un botón que muestra "Comprar", luego spinner, luego "Listo" o "Reintentar".
Señales de uso: if/else por todos lados para estados; peor si te olvidas de uno.
Cuidado: enumera explícitamente los estados válidos; evita estados imposibles (ej. "cargando" y "error" a la vez).

4) Decorator / HOC

Idea simple: agregar capacidades a algo sin tocar su interior.
Para qué sirve: añadir logging, analytics, cache, theming a componentes o funciones.
Cómo se ve: un HOC o Hook que envuelve tu componente y le agrega métricas; un interceptor de Axios que añade un token.
Señales de uso: quieres sumar una función "alrededor" (antes/después) sin modificar el código original.
Cuidado: demasiados decoradores encadenados → difícil saber qué pasa primero.

5) Adapter

Idea simple: hacer que dos APIs distintas se entiendan.
Para qué sirve: cambiar de proveedor sin reescribir tu app.
Cómo se ve: una capa payments.pay(total) que por dentro llama a Stripe o PayPal.
Señales de uso: integras un servicio nuevo cuya API no coincide con tu interfaz actual.
Cuidado: el adaptador traduce, no "arregla" problemas de diseño del proveedor.

6) Facade

Idea simple: ofrecer una cara simple a un sistema complejo.
Para qué sirve: que el equipo use una API sencilla y consistente.
Cómo se ve: VideoService con métodos fáciles (grabar(), detener()) que internamente usa cámara, permisos, compresión, y subida.
Señales de uso: muchos módulos y pasos repetidos; la gente se confunde.
Cuidado: la fachada no debe ocultar errores importantes; propaga mensajes útiles.

7) Composite

Idea simple: tratar igual a un elemento y a un grupo de elementos.
"borrar/renombrar".
Para qué sirve: manejar UIs en árbol (menús, árbol de archivos, escenas).
Cómo se ve: componentes que contienen hijos y delegan acciones como render() a cada uno.
Señales de uso: necesitas hacer la misma operación a un ítem o a una colección.
Cuidado: define bien quién puede tener hijos y quién no.

Backend

8) Singleton

Idea simple: que exista una sola instancia compartida.
Para qué sirve: conexión a BD, cliente Redis, configuración global.
Señales de uso: crear varias instancias rompe el rendimiento o el estado compartido.
Cuidado: evita usarlo como “variable global para todo”; úsalo solo cuando de verdad debe ser único.

9) Factory / Abstract Factory

Idea simple: una fábrica que crea el objeto correcto según el entorno.
Para qué sirve: elegir implementaciones (memoria, Postgres, Dynamo) con la misma interfaz.
Señales de uso: necesitas cambiar de tecnología sin tocar la lógica de negocio.
Cuidado: no escondas dependencias mágicas; documenta cada “producto”.

10) Builder

Idea simple: construir un objeto paso a paso y al final “armarlo”.
Para qué sirve: crear objetos complejos (emails con adjuntos, queries).
Señales de uso: creaciones con muchos parámetros opcionales que se vuelven confusas.
Cuidado: no conviertas algo simple en ritual; si son 2 opciones, no uses Builder.

11) Command

Idea simple: guardar una acción como un objeto para ejecutar, deshacer o encolar.
Para qué sirve: colas de jobs, macros, undo/redo.
Señales de uso: necesitas reintentos, historial o programar la ejecución.
Cuidado: registra el resultado y errores; define bien do/undo.

12) Chain of Responsibility

Idea simple: una cadena de pasos; cada uno decide si maneja o pasa al siguiente.
Para qué sirve: pipelines de validación, seguridad, transformación.
Cómo se ve: middlewares en Express: rate-limit → auth → auditoría → handler.
Señales de uso: la misma petición debe pasar por varios filtros ordenados.
Cuidado: documenta el orden; un paso mal puesto rompe todo.

13) Proxy

Idea simple: un representante que se interpone para añadir algo.
Para qué sirve: cache, control de acceso, lazy-loading, rate limit.
Cómo se ve: un proxy que guarda en memoria respuestas de una API externa.
Señales de uso: muchas llamadas a lo mismo; necesitas ahorrar tiempo o proteger un recurso.
Cuidado: invalida la caché correctamente; evita datos viejos.

14) Template Method

Idea simple: definir el esqueleto de un proceso y permitir "ganchos" para personalizar partes.
Para qué sirve: procesos con estructura repetida (validar → autorizar → ejecutar).
Cómo se ve: controlador base que hace lo común y delega handle() a hijos.
Señales de uso: muchos servicios hacen casi lo mismo con pequeñas variaciones.
Cuidado: no bloquees personalizaciones necesarias; ofrece suficientes hooks.

5. ¿Cómo elegir?

- Se actualiza mucha UI por cambios? → Observer
- Muchas variantes del mismo cálculo? → Strategy
- El comportamiento cambia por etapas? → State
- Quieres sumar algo alrededor sin tocar el núcleo? → Decorator
- Integras un proveedor distinto? → Adapter
- Muchos pasos repetidos y confusos? → Facade
- Estructura en árbol? → Composite
- Única instancia global? → Singleton
- Cambiar de tecnología sin tocar lógica? → Factory
- Objetos con mil opciones? → Builder
- Necesitas deshacer/colar tareas? → Command
- Varios filtros en orden? → Chain
- Necesitas cache o puerta de entrada? → Proxy
- Proceso con esqueleto común? → Template Method

6. **Beneficios**
   - Reutilización, menos acoplamiento, vocabulario común.
   - Sobre-ingeniería si el problema es simple; evita “patronitis”.

7. **Relación con frameworks**

- Frameworks usan patrones:
  - Middlewares (Express) ⇒ *Chain of Responsibility*.
  - Hooks/Context (React) + stores ⇒ *Observer*.
  - Query builders (Knex/Prisma) ⇒ *Builder*.
  - Guards/Interceptors (NestJS) ⇒ *Decorator/Chain*.
- Se aplica patrones dentro del framework para mantener el código desacoplado, testeable y fácil de extender.